

COMPUTING SPECTRUM

EDITION 3.0



IEEE
COMPUTER
SOCIETY



TABLE OF CONTENTS

01 INTRODUCTION

The IEEE Computer Society is a global network of innovators, engineers, and tech enthusiasts dedicated to advancing technology for humanity.

02 IEEE TEAM

Get to know the dedicated members, organizers and mentors of our IEEE Computer Society who work together to bring ideas to life and create a thriving tech community.

03 ARTICLES & IDEAS

Explore a collection of thought-provoking articles written by our members, faculty, and guest contributors.

04 IEEE EVENTS

From workshops and seminars to coding competitions and tech fests, this section captures the spirit of IEEE events.

05 UNTIL NEXT BYTE

As we close this edition with our achievements, we extend our sincere gratitude to our contributors, readers, and the entire IEEE community for being part of this journey.

COMPUTING SPECTRUM

Message from Campus Coordinator

Prof. Dr. Muhammad Shahbaz

It gives me immense pleasure to congratulate the entire team behind the IEEE Magazine. This milestone marks not just a beginning, but a testament to the passion, creativity, and dedication of our students and faculty specially Ms. Fatima who have worked diligently to bring this initiative to life.

This magazine serves as a powerful platform to showcase the talent, innovation, and intellect of our students. I am positive that it will reflect the vibrant academic culture we strive to foster at our campus; one that values curiosity, encourages inquiry, and celebrates achievement. I firmly believe that such student-led initiatives play a critical role in nurturing leadership, collaboration, and communication skills that are essential for professional growth.

I have high hopes and expectations from the IEEE student body. You represent the future of technology, and your contributions will shape not only the future of our university but also the betterment of society at large. I urge you to continue thinking beyond boundaries, embracing challenges, and using your knowledge to make meaningful impacts. Remember, excellence is a journey, not a destination. Strive to uphold the values of integrity, perseverance, and innovation in all that you do. Let this magazine be a beacon of inspiration for others and a symbol of what we can achieve together when we work with purpose and unity. Once again, congratulations to the IEEE Magazine team on this remarkable achievement. May this be the first of many milestones to come.

COMPUTING SPECTRUM

Message from Advisor

Dr. Sadia Tariq

It gives me great pleasure to present this year's edition of the IEEE Computer Society Chapter magazine, marking another year of dedication, learning, and professional excellence. Over the past year, our Chapter has successfully organized a wide range of co-curricular activities, including technical workshops, expert webinars, knowledge-sharing sessions, quiz competitions, and indoor competitions. These initiatives have not only enhanced the technical and professional skills of our students but have also fostered teamwork, leadership, and active student engagement. I extend my sincere appreciation to the Chapter officers, volunteers, faculty members, and all participants whose commitment made these achievements possible.

As we move forward, I encourage every member to continue embracing innovation, collaboration, and lifelong learning while upholding the highest standards of professionalism and ethical conduct. I hope this magazine serves as a testament to our collective accomplishments and inspires future generations to actively contribute to the IEEE Computer Society Chapter. I wish the Chapter continued success in all its future endeavors.

COMPUTING SPECTRUM

Message from Coordinator

Fatima Shahzadi

The IEEE Computer Society (IEEE-CS) chapter continues its momentum of growth and impactful accomplishments, and I remain deeply honored to serve as its Coordinator. Building on the strong foundation laid in our previous years—through workshops, webinars, and expert talks—we have stayed fully in pace with the momentum and initiatives that defined our earlier success. The continuous dedication of our entire core body has been the driving force behind this sustained progress, keeping our community vibrant and forward-looking.

With great pride, I present the 3rd edition of our magazine as we continue our commitment to publishing semi-annually and offering a robust platform for knowledge sharing. This edition marks an extraordinary milestone: while we maintained our previous momentum, we are now taking a monumental leap forward by partnering with a globally recognized platform in quantum computing. This major collaboration elevates our technical scope, giving our members direct access to cutting-edge tools and learning environments in one of today's most transformative fields.

In this and upcoming editions, we will continue to showcase entrepreneurial journeys, research endeavors, and innovative contributions from our students and alumni who are making meaningful impacts within and beyond campus. I extend my heartfelt gratitude and best wishes to the entire IEEE-CS leadership and core team. Through your continued vision, collaboration, and relentless commitment, I am confident our chapter will keep inspiring and shaping the next generation of technology leaders.

INTRODUCTION



COMPUTING SPECTRUM

IEEE Computer Society

The IEEE Computer Society Narowal Chapter is a dynamic and rapidly growing student-led body under the umbrella of the IEEE UET Lahore Narowal Campus Student Branch.

Established with the mission to bridge the gap between academic learning and real-world technological advancements, the chapter serves as a vibrant platform for aspiring computer scientists, engineers, and tech enthusiasts.

Driven by innovation, collaboration, and excellence, the chapter organizes a variety of technical workshops, seminars, hackathons, coding competitions, and industrial collabs to empower students with practical

skills and industry-relevant knowledge. It fosters an inclusive environment where students can explore diverse fields such as artificial intelligence, cybersecurity, software engineering, data science, and more.

As a part of the globally renowned IEEE Computer Society, the Narowal Chapter aims to uphold the highest standards of professionalism and academic integrity. It also encourages leadership, teamwork, and community engagement, equipping members with not just technical proficiency, but also the confidence to lead and make a difference.

With a strong commitment to growth and innovation, the IEEE Computer Society Narowal Chapter continues to inspire a new generation of thinkers, problem-solvers, and changemakers in the field of computing.

COMPUTING SPECTRUM

Department of Computer Science & Engineering

Department of Computer Science and Engineering, Narowal Campus started working in 2014 at UET Lahore, Narowal. Here the students are given the sterling opportunity to get education of highest standards in pleasant and friendly atmosphere, and to make compatible with their means. The students are laudably facilitated with Well-equipped Laboratories, state of the art Computer Labs with Internet Facility, Multimedia Assisted Classrooms, Innovation Center, unlimited Scholarships and consistently Updated Library. One of the most admirable features of Computer Science and Engineering Department at Narowal campus is its Highly Educated Faculty.



COMPUTING SPECTRUM

Department Spotlights

At UET Narowal, our students consistently shine in both academic and co-curricular arenas, making their mark on national stages. In this edition, we proudly spotlight the outstanding achievements of students from Computer Science department who have brought honor to the university by securing top positions in prestigious competitions and representing UET Narowal at the national level.



COMPUTING SPECTRUM



Computing Spectrum

Official Magazine of IEEE Computer Society

Narowal Chapter | Edition 3.0

Computing Spectrum is the official magazine of the IEEE Computer Society Narowal Chapter, created as a platform to showcase the creativity, knowledge, and technological insight of students, faculty, and professionals in the field of computer science and engineering.

Launched with the vision to inform, inspire, and innovate, this magazine serves as a reflection of the dynamic and ever-evolving world of computing. From thought-provoking articles on emerging technologies like artificial intelligence, machine learning, cybersecurity, and data science, to interviews, student research, event highlights, and creative tech content, Computing Spectrum offers a diverse range of perspectives and voices. This inaugural edition marks an important milestone for our chapter and our university, providing students with the opportunity to express their ideas, share their achievements, and connect with a wider community of tech enthusiasts.

Each page represents the dedication and collaborative spirit of our contributors, and we hope this magazine becomes a lasting source of inspiration and pride for current and future generations of innovators at UET Lahore Narowal Campus.

MY TEAM



IEEE TEAM



Dr. SADIA TARIQ
ADVISOR

Guides the team with wisdom, experience, and a vision for excellence.



FATIMA SHAHZADI
COORDINATOR

Coordinates efforts and ensures smooth communication across the team.



HASEEB AHMAD
MENTOR

Provides mentorship and technical guidance to help team grow and achieve milestones.



ALIZA JAVED
PRESIDENT

Leads the chapter with vision and empowers the team to achieve excellence through collaboration.



WAQI UL HUSNAIN
VICE PRESIDENT

Supports the President and ensures the successful execution of organizational goals.



GAGAN SOHAIL
GENERAL SECRETARY

Manages official documentation and ensuring efficient coordination across the executive team.



MAHAM PERVAIZ
TREASURER

Manages the financial planning, and expense management for events and organizational activities.



FARAZ MUNIR KHAN
MANAGING DIRECTOR

Oversees operations and ensures the team's resources are used efficiently.

DIRECTOR TEAM



ABDUL QAYYUM

DIRECTOR

DEPARTMENT OF
TECH



IZBA MAHAM

CO DIRECTOR

DEPARTMENT OF
TECH



ABIHA

DIRECTOR

DEPARTMENT OF
MEDIA & DESIGN



MAIDA ZAHAD

CO-DIRECTOR

DEPARTMENT OF
MEDIA & DESIGN



ABDULLAH

GENERAL MANAGER

IEEE COMPUTER
SOCIETY



HAMZA GILLANI

SOCIAL MEDIA MANAGER

IEEE COMPUTER
SOCIETY

EDITORIAL TEAM



HASSAN ALI

ALUMNI

Editorial Advisor



ALIZA JAVED

PRESIDENT

Editorial Designer



ABIHA

DIRECTOR

Editorial Coordinator



ABDULLAH

GENERAL MANAGER

Content Editor

ARTICLES & IDEAS



Error Rates in Quantum Computing

Dr.SadiaTariq

“The power of a quantum computer is limited not only by the number of qubits it has, but by how accurately those qubits can preserve information.”

Quantum computing is one of the most exciting technologies of the 21st century. By exploiting quantum mechanics, it has the potential to revolutionize cryptography, drug discovery, optimization, artificial intelligence, and materials science.

However, despite rapid progress, modern quantum computers are still prone to errors. These errors reduce the reliability of calculations and remain the primary obstacle to building large-scale practical quantum computers.

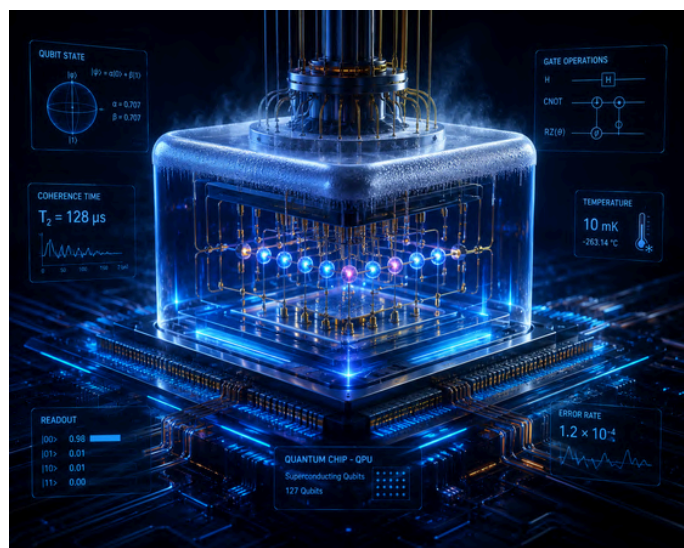


FIGURE 1. A superconducting quantum processor at 10 mK — qubit state, gate operations, and readout fidelity monitored

HOW QUANTUM INFORMATION IS STORED

Classical computers store information as bits (0 or 1), whereas quantum computers use qubits. Qubits can exist in superposition and become entangled, enabling parallel computation. One common implementation is the trapped-ion quantum computer, where individual charged atoms are suspended using electromagnetic fields inside an ultra-high vacuum. Carefully controlled laser pulses manipulate these ions to perform quantum operations.

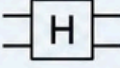
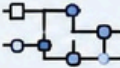
SOURCES OF ERRORS

Errors arise from bit-flips, phase-flips, decoherence, gate imperfections and measurement errors. Even tiny amounts of heat, electromagnetic interference or vibration can disturb fragile quantum states. As the number of quantum operations increases, these small errors accumulate and reduce computational accuracy.

REDUCING ERRORS

Researchers improve qubit quality, shielding, cooling systems and control electronics. Quantum Error Correction (QEC) spreads information across many physical qubits to create a reliable logical qubit. Surface-code techniques are currently considered among the most promising approaches for fault-tolerant quantum computing, although they require many physical qubits.

KEY STATICS: ERROR RATE IN QUANTUM COMPUTING

Metric	Typical Value	What It Means
 Two-qubit gate error rate	0.1 – 1% (99 – 99.9% fidelity)	Every two-qubit gate operation has a 0.1% to 1% chance of producing an error.
 Circuit with 1,000 operations	~1 expected error (at 0.1% gate error rate)	As the number of operations increases, errors accumulate rapidly.
 Physical qubits per logical qubit	100 – 1,000+	Hundreds to thousands of physical qubits may be required to create one reliable logical qubit using error correction.
 Coherence time (T_2)	~100 μs to a few seconds (technology-dependent)	Qubits lose their quantum state over time. Longer coherence time helps, but longer computations face higher error risk.
 Error rate scaling with more qubits	Increases significantly	More qubits mean more noise sources, crosstalk, and decoherence, making error management harder.

CONCLUSION

Reducing error rates is the key challenge for practical quantum computing. Continued advances in hardware, quantum algorithms and error-correction techniques are steadily improving reliability. As these technologies mature, fault-tolerant quantum computers will become capable of solving problems that are impossible for today's classical machines.

KEY TAKEAWAYS

- Qubits are fragile and easily disturbed.
- Trapped ions and superconducting circuits are leading qubit technologies.
- Quantum Error Correction is essential for reliable quantum computers.
- Lower error rates will enable practical large-scale quantum applications.

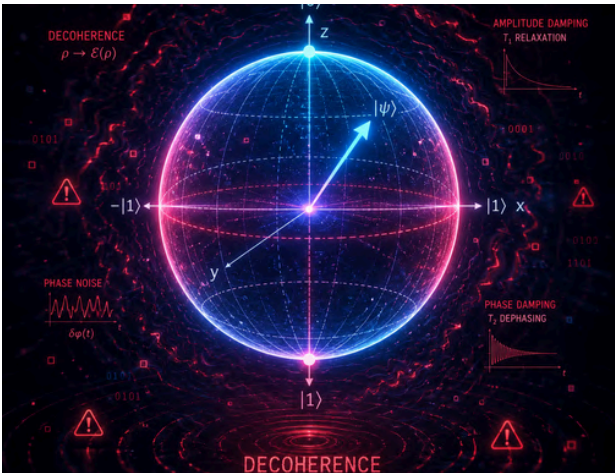


FIGURE 2. The Bloch sphere under decoherence — mplitude damping and phase noise erode a qubit's quantum state over time.

REFERENCES

1. Michael A. Nielsen and Isaac L. Chuang, Quantum Computation and Quantum Information, Cambridge University Press, 2010.
2. John Preskill, ‘Quantum Computing in the NISQ Era and Beyond’, Quantum, 2018.
3. Nature Reviews Physics — Quantum Error Correction.

Orchestrating Insights: The Rise of AI Agents

Haseeb Ahmad

“An agent doesn’t just answer questions — it reasons about a goal, makes a plan, and takes action to complete a task.”

What if a computer program could do more than answer questions and actually complete tasks for

you? That idea is at the heart of an AI agent. An AI agent is a program that can reason about a goal, make a plan, and take actions with limited human guidance to complete a task. This is different from a workflow, where steps and rules are mostly fixed in advance (static), like following a recipe, while an agent has more freedom to make dynamic decisions based on the situation. This is also the difference between agentic AI and automation: automation usually follows predefined instructions, while agentic AI can adapt its actions to reach a goal.

To understand why agents matter, we first need to understand a large language model, or LLM. An LLM is the kind of AI that writes text and answers questions, like the chatbots many of us already use. Inside an LLM, pieces of text called tokens are converted into numerical representations known as embeddings, and text with similar meanings lands closer together in that space. These embeddings are then processed by a Transformer architecture, which uses attention to focus on the most relevant earlier information when understanding or generating text.



FIGURE 1. Overview of an autonomous AI agent ecosystem combining memory, goal-driven execution, and security considerations."

INSIDE THE AGENT LOOP

MEMORY: SHORT-TERM & LONG-TERM

One interesting part of many agents is how they remember things, using Short-Term Memory (STM) and Long-Term Memory (LTM). STM is like remembering what a friend said five minutes ago, useful only for the current task, while LTM works like remembering something important from last week, saved for later use. Together, they help an agent avoid starting from scratch every time.

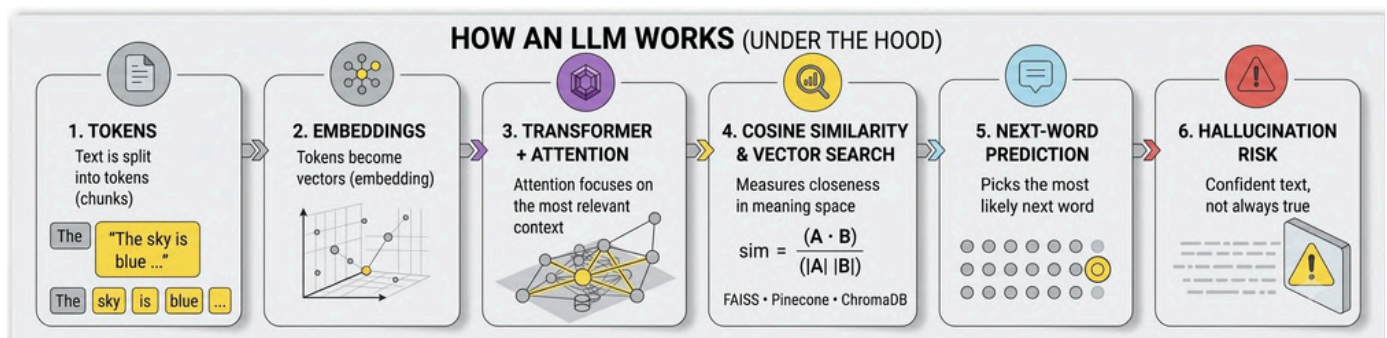


FIGURE 2. The step-by-step workflow of a Large Language Model from raw text tokenization to output generation.

HUMAN-IN-THE-LOOP

Memory alone does not make an agent safe to trust. This is where human-in-the-loop comes in: a real person can review, approve, or correct important decisions before they are accepted, so mistakes are caught early.

INSIDE AN LLM: HOW IT WORKS

Step	What Happens
Tokens & Embeddings	Text is split into tokens and converted into numerical embeddings,
Transformer + Attention	Attention focuses on the most relevant earlier context while the model
Cosine Similarity	$\text{similarity} = (A \cdot B) / (A B)$ — measures closeness in meaning space; a value
Next-Word Prediction	The LLM predicts the statistically most likely next word based on
Hallucination Risk	Confident-sounding text that isn't grounded in fact, since the model has no

CONCLUSION

For an agent to be truly useful, it also needs outside tools, like searching the web or reading files. This is where MCP, or Model Context Protocol, helps: an open standard giving AI applications a common way to connect with supported tools and data sources. To build agents, developers use frameworks like LangGraph and CrewAI, which provide structures for planning, memory, and tool use. Platforms like n8n are different — they are mainly designed for automation, although they can also support simpler forms of agentic behavior.

Agents are already used in real life, and they are also speeding up how software gets built. Customer support agents, like Intercom's Fin, answer queries on their own. Research agents, like Perplexity and OpenAI's Deep Research, summarize findings from multiple sources in seconds.

Coding agents go further, reading a codebase and helping with bug fixes, using tools like Claude Code, Codex, GitHub Copilot, and Cursor.

KEY TAKEAWAYS

- Agents differ from workflows by making dynamic decisions rather than following fixed, predefined steps.
- Embeddings, attention, and cosine similarity let an LLM reason about meaning — but next-word prediction can still produce hallucinations.
- STM/LTM memory plus human-in-the-loop review make agents safer to trust with real tasks.
- Member of Technical staff at Anthropic Lamis Mukta described “Dreaming” — an out-of-band process that reviews past sessions to curate an agent's memory, always with human approval before changes go live.
- Prompt injection remains a real risk: a hidden instruction in a webpage or document can trick an agent into leaking data, so agents must be designed and monitored carefully.

TOOLS & PLATFORMS MENTIONED

- FAISS, Pinecone, ChromaDB — embedding storage & similarity search
- LangGraph, CrewAI — agent-building frameworks
- n8n — automation platform with simple agentic features
- MCP (Model Context Protocol) — standard for connecting tools & data sources
- Claude Code, Codex, GitHub Copilot, Cursor — coding agents
- Intercom Fin, Perplexity, OpenAI Deep Research — support & research agents

The Quantum Deadline May Have Moved Closer

Talha Amir

“Attackers can collect encrypted traffic now and keep it until a powerful quantum computer arrives — harvest now, decrypt later.”

W*hat if someone copied your most private encrypted data today, locked it away, and simply waited for the key to be invented?*

This is already possible. Attackers can collect encrypted traffic now and keep it until a powerful quantum computer arrives. The strategy is called “harvest now, decrypt later.” Medical records, trade secrets, state communications, and other long-lived information could still be valuable when that day comes.

The proposed defence is post-quantum cryptography, or PQC. Despite the name, PQC does not require a quantum computer — it is ordinary software running on conventional phones, laptops, servers, and chips. What changes is the mathematics underneath.

Today, much of the internet depends on mathematical traps. RSA hides secret information behind the difficulty of factoring enormous numbers. Elliptic-curve cryptography uses a different problem that is also extremely difficult for conventional computers. A sufficiently capable quantum computer running Shor's algorithm could solve both efficiently.

PQC replaces them with puzzles believed to resist both classical and quantum attacks —

many built on structured lattices (recovering a hidden point inside an immense, distorted grid), others on one-way hash functions.

WHY MIGRATION MAY BE MORE URGENT

THE ESTIMATE SHRANK, NOT THE HARDWARE

A 2019 estimate co-authored by Google researcher Craig Gidney suggested that factoring an RSA-2048 number in eight hours would require about 20 million noisy physical qubits. In 2025, Gidney produced a much smaller estimate: by combining improved quantum algorithms and error-correction techniques, RSA-2048 could be factored in less than a week using fewer than one million noisy qubits.

THE MACHINE STILL DOESN'T EXIST

That machine does not exist. The estimate assumes extremely low gate errors, rapid operations, and a large fault-tolerant architecture — major engineering problems remain in fabrication, cooling, control, and scaling. The paper offers no date for the fall of RSA. Its warning comes from the direction of travel: researchers cut the estimated hardware requirement by roughly twenty times through better methods alone.

NIST'S PQC STANDARDS (FINALIZED 2024)

Standard	Purpose
ML-KEM	Lets two parties establish a shared secret for encryption, replacing RSA/ECC key exchange.
ML-DSA	General-purpose digital signatures built on structured lattices.
SLH-DSA	A signature alternative based on hash functions, for extra design diversity.

CONCLUSION

Migration has its own clock. Cryptography is embedded inside browsers, certificates, payment systems, industrial equipment, chips, and old software — finding and replacing it can take a decade or longer. Larger PQC keys and signatures can affect performance, networks, and constrained devices, so deployments need careful testing. Some systems may initially combine established and post-quantum algorithms as a safety measure.

The practical question is therefore not “When will quantum computers break encryption?” Ask how long the data must remain secret, how long migration will take, and whether those periods extend into the quantum-risk window. The machine that threatens RSA may still be years away — but for information worth keeping, the migration deadline could be much closer.



FIGURE 1. Lattice-based structures form the mathematical foundation of post-quantum cryptography.

KEY TAKEAWAYS

- “Harvest now, decrypt later” means encrypted data collected today is already at risk if it must stay secret for years.
- PQC is ordinary software — no quantum computer is needed to run it, only new math underneath (lattices, hash functions).
- NIST finalized three deployable PQC standards in 2024: ML-KEM, ML-DSA, and SLH-DSA.
- Research, not new hardware, cut the estimated qubits to break RSA-2048 by roughly 20× between 2019 and 2025.

REFERENCES

- Craig Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits,” Google Research / arXiv, 2025.
- NIST, “What Is Post-Quantum Cryptography?” updated February 27, 2026.
- NIST, “Announcing Approval of Three Federal Information Processing Standards (FIPS) for Post-Quantum Cryptography,” August 13, 2024.
- NIST IR 8547, “Transition to Post-Quantum Cryptography Standards,” initial public draft, November 2024.
doi.org/10.6028/NIST.IR.8547.ipd
- CISA, NSA, and NIST, “Quantum-Readiness: Migration to Post-Quantum Cryptography,” 2023.

Design Database for Data-Intensive APIs

Abdullah Zahid

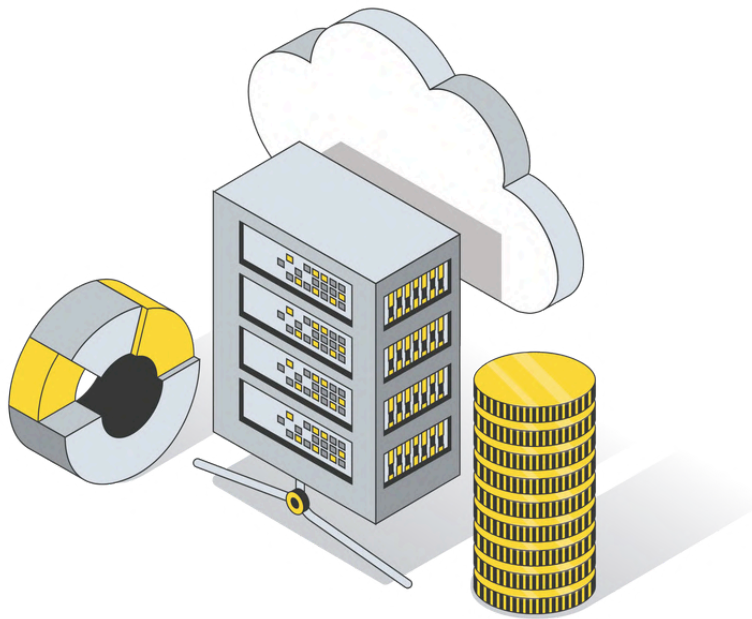


FIGURE 1. Architecting efficient database layouts to seamlessly power high-throughput, data-intensive APIs.

When I first started working with databases in real projects, I thought designing a database just meant creating tables and connecting them to an API. It took actual production traffic, slow endpoints, and a few embarrassing bugs to understand that database design is a process, not a one-time task. And the best way I've found to explain it to someone just starting is to walk through it the same way we build software, stage by stage, because that's exactly how it plays out in real projects too.

01. PLANNING

Everything begins at the planning stage, before a single line of code is written. This is where you sit down and think about what data your application actually needs, how different pieces of information relate to each other, and how they'll be accessed later. A lot of junior developers, myself included back then, jump straight into creating tables without thinking about relationships properly. But spending time here, figuring out your schema and how entities connect through

through foreign keys, saves you from painful restructuring later. The goal isn't to make everything perfectly normalized just for the sake of theory; it's to design something that makes sense for how your application will actually read and write data.

02. DEVELOPMENT — INDEXING

Once you move into development and start writing actual queries, indexing is usually the first performance concept you'll run into. In simple words, an index helps the database find data faster, kind of like an index page in a book helping you jump straight to a topic instead of flipping through every page. But here's the part most beginners miss, indexes only help if they match the way you're actually querying the data. If your query filters by user ID first and then sorts by date, your index should follow that exact sequence. Randomly indexing columns because they seem important can actually slow down your write operations without giving you any real benefit.

CONNECTION POOLING

As your API starts talking to the database more frequently, you'll come across something called connection pooling. Think of it like every time your API needs to talk to the database, it needs a connection, kind of like a phone line. Opening a new phone line for every single request is slow and resource-heavy. Instead, connection pooling keeps a set of ready-to-use connections open and reuses them, so your API doesn't waste time setting up new connections constantly. This becomes especially important once your application scales and multiple users are hitting your API at the same time.

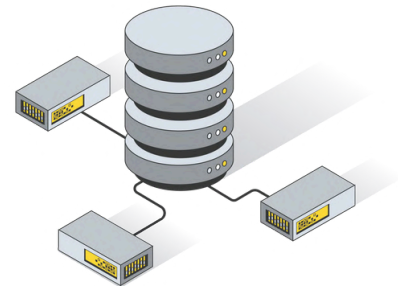


FIGURE 2. Optimizing database communication through active connection pooling

03. TESTING — THE N+1 PROBLEM

Then comes testing, and this is where a very sneaky problem usually shows up, called the N+1 query problem. Imagine you fetch a list of ten students, and then for each student you run a separate query to get their marks. That's eleven queries when it could've been just one or two done smartly. It doesn't look like a big issue when you're testing with small dummy data, but the moment real data comes in, your API slows down badly. Learning to spot this early, usually by checking your query logs during testing, is one of those lessons that separates a beginner from someone who actually understands performance.

PAGINATION & QUERY LIMITS

This is also the stage where you start thinking seriously about pagination and query limits. Early on, it's easy to write a query that just fetches everything, all records, no restrictions. It works fine with ten rows in your database. But in production, tables can have millions of rows, and returning all of that in one API call can crash your server or completely freeze the response.

Pagination simply means breaking that data into smaller chunks, like showing twenty results per page instead of ten thousand at once. Alongside that, putting limits on expensive or time-consuming queries protects your database from being overloaded, whether by accident or by someone misusing your API.

04. DEPLOYMENT — CACHING

When your project reaches the deployment stage, caching becomes one of the most impactful things you can add. Not all data changes frequently; some information gets requested again and again with barely any change, like a list of categories or popular products. Instead of hitting the database every single time for the same data, caching stores that result temporarily so it can be served instantly on the next request.

I still remember the first time I added caching to a slow endpoint and watched the response time drop from almost two seconds to under a hundred milliseconds; it genuinely feels like magic the first time you see it.

PRODUCTION — READ REPLICAS

Once your API actually goes live and starts getting real users, read replicas start making a lot of sense. In simple terms, a read replica is a copy of your main database that only handles read operations, while the main database focuses on write operations like inserting or updating data. This separation means your important write operations don't get delayed because someone is running heavy read queries elsewhere. It's a common pattern in production systems and something juniors usually only learn about once traffic actually starts growing.

05. MAINTENANCE — AUTO-SCALING

Finally, in the maintenance and monitoring stage, auto-scaling becomes your safety net. Traffic isn't always predictable; a sudden spike during a sale, an exam result announcement, or a viral social media post can bring way more users than usual. Manually adding servers or resources every time this happens isn't realistic. Auto-scaling allows your infrastructure to automatically adjust based on incoming traffic, scaling up during peak times and scaling back down when things calm down, so your system stays stable without constant manual intervention.

THE BIG PICTURE

Looking back at everything I've learned in these two years, database design for data-intensive APIs isn't about knowing one big trick; it's about understanding these small decisions and knowing exactly where they fit in your development process. Once you start seeing your database design through this lifecycle lens - planning, developing, testing, deploying, and maintaining - it stops feeling overwhelming and starts feeling like a natural part of building good software.

Quantum Machine Learning

Maham Pervaiz

What is quantum machine learning?

Quantum machine learning is a research area that explores the interplay of ideas from quantum computing and machine learning. For example, we might

want to find out whether quantum computers can speed up the time it takes to train or evaluate a machine learning model. On the other hand, we can leverage techniques from machine learning to help us uncover quantum error-correcting codes, estimate the properties of quantum systems, or develop new quantum algorithms.

QUANTUM COMPUTERS AS AI ACCELERATOR

The limits of what machines can learn have always been defined by the computer hardware we run our algorithms on—for example, the success of modern-day deep learning with neural networks is enabled by parallel GPU clusters.

Quantum machine learning extends the pool of hardware for machine learning by an entirely new type of computing device—the *quantum computer*. Information processing with quantum computers relies on substantially different laws of physics known as *quantum theory*.

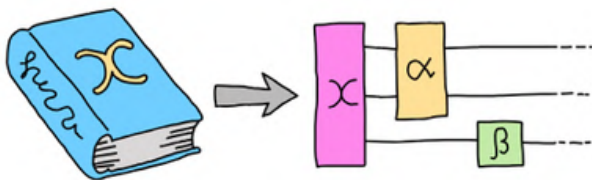


FIGURE 1. Translating classical data and algorithms into parameterized quantum circuit operations.

MACHINE LEARNING ON QUANTUM DEVICES

Quantum computers are here and accessible today, but they are **noisy**. When they first became available, this motivated a growing interest in the search for quantum machine learning applications on noisy devices, the so-called Noisy Intermediate-Scale Quantum (NISQ) era. We can understand these devices as special-purpose hardware like Application-Specific Integrated Circuits (ASICs) and Field-Programmable Gate Arrays (FPGAs), which are more limited in their functionality.

However, after an initial period of optimism, it became clear that QML applications on near-term hardware do not stand out in performance compared to classical machine learning algorithms, and there is a renewed focus on **understanding** and **benchmarking** such systems.

Alongside near-term devices, there is also interest in exploring QML using ideal, universal quantum computers ("fault-tolerant quantum computers, or FTQCs") which are still under heavy development. This utilizes various tools and approaches, such as abstract learning theory, groups, and symmetries.

- **Present:** QML is promising but limited by current noisy hardware.
- **Future:** More advanced fault-tolerant quantum computers could make QML much more powerful.

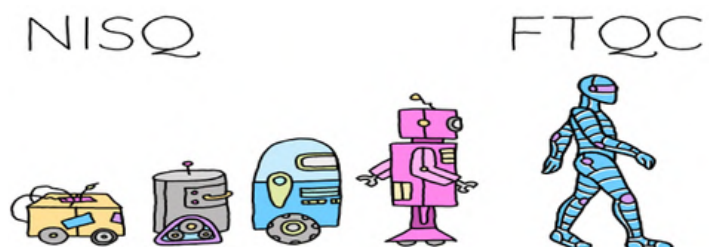
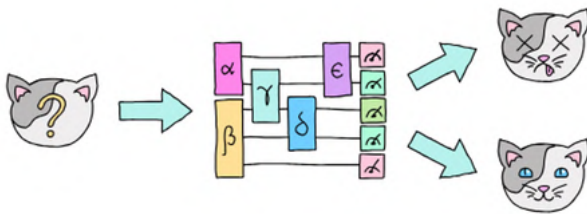


FIGURE 2. The technological evolution from near-term NISQ devices, fault-tolerant quantum computing (FTQC)

USING QUANTUM COMPUTERS

In the modern viewpoint, quantum computers can be used and trained like neural networks. We can systematically adapt the physical control parameters, such as an electromagnetic field strength or a laser pulse frequency, to solve a problem.

For example, a trained circuit can be used to classify the content of images, by encoding the image into the physical state of the device and taking measurements



USING QUANTUM COMPUTERS

But the story is bigger than just using quantum computers to tackle machine learning problems. Quantum circuits are differentiable, and a quantum computer itself can compute the change in control parameters needed to become better at a given task.

Differentiable programming is the very basis of deep learning, implemented in software libraries such as TensorFlow and PyTorch. Differentiable programming is more than deep learning: it is a programming paradigm where the algorithms are not hand-coded, but learned.

Similarly, the idea of training quantum computers is larger than quantum machine learning. Trainable quantum circuits can be leveraged in other fields like **quantum chemistry** or **quantum optimization**.

It can help in a variety of applications such as the design of quantum algorithms, the discovery of quantum error correction schemes, and the understanding of physical systems.

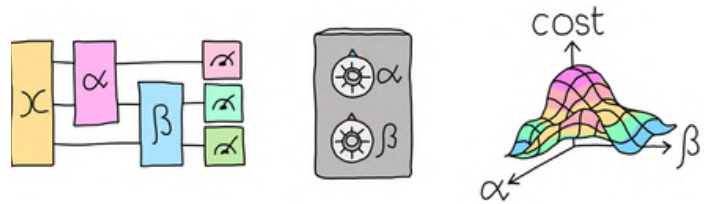


FIGURE 3. Classifying input data through a trained quantum circuit using quantum neural network principles.

REFERENCES

1-PennyLane Blog, "How to Start Learning Quantum Machine Learning," October 13, 2021.

<https://pennylane.ai/blog/2021/10/how-to-start-learning-quantum-machine-learning/>

2- Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd, "Quantum Machine Learning," *Nature*, Vol. 549, pp. 195–202, 2017.

<https://www.nature.com/articles/nature23474>
<https://doi.org/10.1038/nature23474>

A landmark review of Quantum Machine Learning covering its foundations, algorithms, and applications.

3. IBM Quantum Learning, "Introduction to Quantum Machine Learning," updated regularly.

<https://quantum.cloud.ibm.com/learning/en/courses/quantum-machine-learning/introduction>

How Machine Learning Models Really Learn

Gagan Sohail

Machine learning has become one of the most important technologies behind modern artificial intelligence. It powers applications such as ChatGPT, Google Gemini, Netflix recommendations, Amazon product suggestions, fraud detection, self-driving cars, and medical diagnosis systems. Although these systems appear intelligent, they all learn through the same structured training process. From collecting data to deploying a trained model, every stage plays an important role in building an accurate and reliable AI system.

BUILDING THE LEARNING PROCESS

The process begins with data because every machine learning model learns from examples.

Companies such as OpenAI, Google DeepMind, and Meta collect large amounts of publicly available, licensed, and human-created data to train their models. Before training starts, the data is cleaned by removing duplicates, fixing missing values, filtering low-quality content, and converting it into a format the model can understand. For language models, this includes breaking text into tokens. Tools such as Pandas, NumPy, and Hugging Face Datasets are widely used to prepare data before training.

After the data is ready, the model is created with randomly initialized weights and biases. At this stage, the model has no knowledge and its first predictions are usually incorrect. Weights determine how much influence each input has on the prediction, while the bias allows the model to adjust its output more accurately instead of always starting from zero.

The model then compares its prediction with the correct answer using a loss function, which measures how much error was made. A high loss means the prediction is far from the expected result, while a lower loss indicates that the model is improving. The loss provides the feedback needed for learning because, without it, the model cannot determine whether it is making progress.

The next stage is learning. Using backpropagation, the model calculates how much each weight contributed to the error. An optimizer then updates those weights so future predictions become more accurate.

Popular optimizers such as SGD, Adam, and AdamW are designed to improve learning speed and stability. This prediction, error calculation, and weight update cycle repeats thousands or even millions of times until the model learns meaningful patterns from the data.

TRAINING MODELS AT SCALE

To perform these calculations efficiently, developers use deep learning frameworks such as PyTorch and TensorFlow. Training is accelerated by NVIDIA GPUs, CUDA, and Google's Tensor Processing Units (TPUs), which can process billions of calculations in parallel.

Largescale training also depends on technologies such as Docker, Kubernetes, MLflow, Weights & Biases (W&B), and cloud platforms including Microsoft Azure, Google Cloud Platform (GCP), and Amazon Web Services (AWS) for managing experiments, scaling resources, and deploying models.

Modern AI companies train models on an enormous scale. OpenAI trains GPT models using thousands of GPUs, while Google DeepMind uses TPU supercomputers to develop the Gemini family.

Meta trained its Llama models on clusters containing tens of thousands of GPUs. In 2025, Moonshot AI introduced Kimi K2, a Mixture-of-Experts model with approximately 1 trillion parameters, while activating only about 32 billion parameters for each token, making the model significantly more efficient. Training frontier AI models can cost tens to hundreds of millions of U.S. dollars, mainly because of the massive computing infrastructure, electricity, storage, and engineering resources required.

Once training is complete, the model is evaluated using data it has never seen before. This step verifies whether the model has learned general patterns instead of memorizing the training data. Metrics such as Accuracy, Precision, Recall, and F1-Score are commonly used to measure realworld performance before deployment.

After successful evaluation, the trained model is deployed so users can access it through websites, mobile applications, APIs, or cloud services. Products such as ChatGPT, Google Gemini, Microsoft Copilot, Netflix, YouTube, and Amazon rely on deployed machine learning models to provide intelligent services to millions of users every day.

MODEL SHARING & THE FUTRE OF AI

The final stage involves deciding how much of the trained model will be shared with the public. Some companies release open-weight models, where only the trained weights are available. Developers can run or fine-tune these models, but the original datasets, preprocessing methods, and training code remain private. Meta's Llama models are among the best-known examples of this approach.

Other organizations choose a fully open-source strategy by releasing the model architecture, source code, training scripts, evaluation methods, and, when possible, the datasets required to reproduce the entire training process.

Projects from organizations such as Allen AI, EleutherAI, and many models hosted on Hugging Face follow this philosophy, promoting transparency, research, and collaboration.

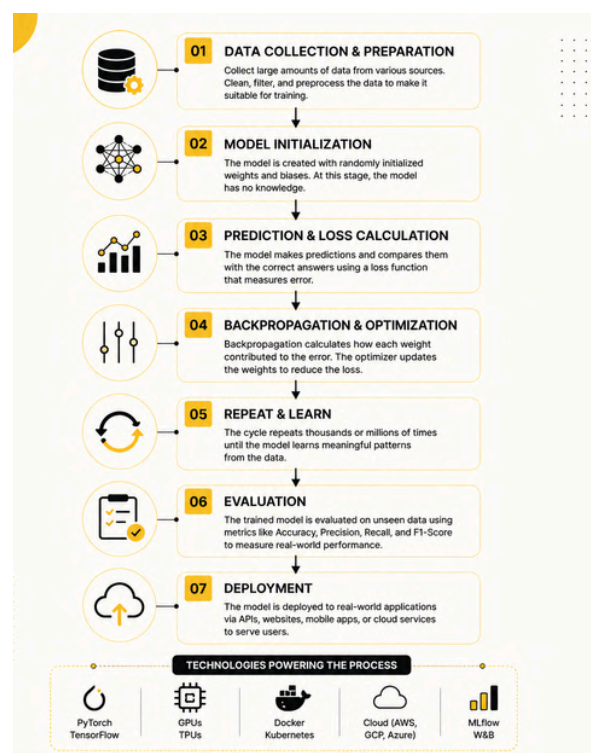


FIGURE 1. The Machine Learning Model Training Pipeline

CONCLUSION

Machine learning is a continuous process that starts with collecting and preparing data, continues through prediction, error measurement, optimization, and evaluation, and ends with deployment and model sharing.

Whether the model is built for a university project or developed by companies such as OpenAI, Google DeepMind, Meta, or Moonshot AI, the core learning process remains the same. The difference lies in the amount of data, computing power, engineering expertise, and technologies used to train these advanced AI systems.

From Pixels to Diagnosis: How Computer Vision Is Transforming Healthcare

Usama Jabar

I*magine you are a radiologist.* You must inspect many MRI, CT, and X-ray scans daily.

One tiny abnormality among thousands of details could reveal an early-stage disease. Your expertise remains essential, but heavy workloads and subtle differences between scans can make abnormalities easy to miss. This is where computer vision comes in, not to replace radiologists, but to assist them, analyzing medical images, highlighting suspicious regions, and helping professionals work faster and decide with more confidence.

THE EVOLUTION OF COMPUTER VISION

Computer vision is a branch of AI that lets machines interpret images and videos, with roots going back to the early years of AI. Frank Rosenblatt built the perceptron in 1958, the simplest type of artificial neuron and the basic building block of neural networks, an early system that could recognize simple visual patterns. In 1966, MIT's Summer Vision Project tried to build a working visual system in a single summer. It badly underestimated the task, but still became an important milestone in the field.

Early computer vision systems relied on manually written rules, searching for edges, shapes, colors, or brightness changes. They worked in controlled settings, but struggled once image quality, anatomy, or lighting changed, and could not adapt, making them unreliable for real medical images.

DEEP LEARNING IN MEDICAL IMAGING

Deep learning changed this. Convolutional Neural Networks, or CNNs, are neural networks built mainly for image processing.

They learn visual patterns directly from training images, instead of depending on human-written rules, and cut training parameters sharply by reusing the same small filter across the image rather than a separate weight per pixel, a technique called weight sharing.

Architectures like U-Net go further: they can outline structures pixel by pixel, a process called segmentation, useful when doctors measure a tumor, trace damaged tissue, or separate an organ from its surroundings. Segmentation accuracy is often measured using a simple formula, Intersection over Union:

$$\text{IoU} = (\text{Area of Overlap}) / (\text{Area of Union})$$

A higher IoU means the model's outline matches the real tissue boundary more closely.

Today, computer vision spans many healthcare tasks. It flags suspicious areas in mammograms, chest X-rays, retinal photographs, MRI scans, and CT images. It outlines tumors and organs for surgery or radiotherapy planning, compares scans taken months apart, and flags cases needing urgent attention.

AI IN HEALTHCARE TODAY

This is no longer confined to research labs: by 2026, the FDA had authorized over 1,500 AI-enabled medical devices, roughly 76% in radiology. Companies like GE Healthcare, Siemens Healthineers, Philips, and Aidoc lead in bringing these tools into hospitals. Still, these systems support professionals; they are not meant to replace clinical judgment.

THE FUTURE OF AI-ASSISTED DIAGNOSIS

Computer vision keeps evolving. Vision Transformers apply self-attention and the transformer architecture, originally built for language processing, to tasks like image classification, splitting an image into small patches and analyzing relationships between distant regions for wider visual context.

More recently, Meta AI, led by Chief AI Scientist Yann LeCun, introduced I-JEPA, the Image-based Joint-Embedding Predictive Architecture: a non-generative, self-supervised model that learns high-level semantic image representations.

Instead of learning only from labeled images or rebuilding every missing pixel, I-JEPA hides parts of an image and predicts the abstract features of the hidden region, not its exact colors.

This could matter a lot in medicine. Hospitals often hold large collections of scans, but labeling them accurately takes specialist time.

A JEPA-based model could first learn general patterns from unlabeled scans, then be adapted for tasks like tumor classification or segmentation: a promising direction, but not yet a routine replacement for established clinical systems.

CONCLUSION

Computer vision still has real limits. Models can make wrong predictions, inherit bias from unrepresentative datasets, and perform worse in a hospital different from the one they were trained in. Patient privacy and data security must be protected at every step.

The future here should not be framed as machines replacing doctors: its real promise lies in combining computational speed and consistency with human experience, responsibility, and care.

The New Entry-Level: How AI Agents Are Changing CS Careers

Azhar Hayat



FIGURE 1. Navigating the shifting landscape of computer science careers in the era of AI agents.

Spend five minutes on any tech forum and you'll see the same worried posts: AI is taking all the junior developer jobs, and the golden age of software engineering is over.

The truth is more complicated, and much less scary. The job market for software engineers is not dead. Global software development is still growing. What's changing is the definition of an "entry-level engineer." The days of landing a well-paid job just because you can build a basic React component are fading.

AI tools have made simple code generation cheap and easy.

AI CODING ASSISTANT, THE NEW REALITY

Companies no longer need people to act as human typists. Instead, they're looking for a new kind of junior developer: someone who can direct the AI.

To understand where the jobs are going, it helps to look at how senior engineering teams work today.

Tools like Anthropic's Claude Code and OpenAI's Codex have moved far beyond the simple autocomplete extensions of a few years ago. Claude Code runs directly from the developer's terminal. It can map out a project's structure, run shell commands, and edit multiple files at once to fix complex bugs.

THE RISE OF AI CODING AGENTS

OpenAI Codex has grown into a full platform that can read a ticket written in plain language, work in its own cloud environment, and handle large amounts of routine work in the background.

In visual editors like Cursor or Devin Desktop, the AI predicts edits across the codebase before the developer finishes typing.

They use a new generation of coding agents that can act on their own.

It can even take a broad goal, read the relevant API documentation, and build the feature from start to finish. Most professional developers now use these agents every day to produce boilerplate code, set up projects, and write predictable functions.

But here's the catch, and it's why human engineers still matter: AI models make things up. They introduce subtle logic errors, drift away from the intended design, and often produce code with known security flaws.

Since an AI agent can write a working draft in seconds, the skill that's now scarce and valuable isn't writing the draft. It's judging whether that draft is correct, secure, and able to scale.

Future engineers will spend less time memorizing syntax and more time guiding AI models, defining hard problems, and carefully checking what the AI produces. The role is shifting from typist to validator

THE ROLE OF FUTURE ENGINEERS

So how do you stand out to hiring managers right now? Start by treating Python or C++ as the baseline, not the goal.

Spend less time grinding algorithm puzzles and more time learning how systems fit together, how APIs, databases, and microservices connect into designs that can grow.

Then, learn to be a careful validator. Companies worry about AI security leaks and bad code reaching production, so a junior developer who can write thorough unit tests for AI-generated code and check for edge cases is worth a lot.

Finally, build with AI instead of avoiding it. Use these tools openly to attempt bigger projects.

A portfolio project that connects to a Large Language Model through an API, or uses an agent to process large datasets, will impress a hiring manager far more than a hand-coded to-do list app or the university website.

CONCLUSION

In the end, AI is not here to replace you. It's a tool, much like the shift from punch cards to modern programming languages. The market is pushing out developers who can only type predictable code, but it's wide open for engineers who can solve problems and manage complex systems.

Change how you think, learn to direct the AI, and you'll be exactly what the tech industry wants from its next generation of engineers.

How AI Agents Are Turning Every Developer Into an Orchestrator

Muhammad Abdullah

“Somewhere in an enterprise right now, an AI agent just closed a support ticket, updated a database, and notified a customer — without a single human clicking a button. Nobody in the building noticed. That's exactly the point.”

For a decade, “AI” meant a chatbot that answered your question and stopped.

You asked, it responded, the conversation ended. That era is over.

FROM TOOL TO TEAMMATE

The old mental model was simple: software is a tool, humans are the operators. That model is breaking down in real time.

In 2026, AI agents don't just answer — they act. They plan multi-step tasks, call tools, make decisions, and complete entire workflows with little to no human supervision.

Analysts are calling 2026 the year AI agents began running entire business operations — proactively completing multi-step tasks, making decisions, and collaborating with other agents to reach complex goals.

This isn't a research demo anymore. Industry trackers report that AI agents have moved out of the lab and into production, with enterprise applications expected to have agents embedded in roughly 40% of them by the end of 2026 — up from under 5% just a year earlier.



FIGURE 1. That's not incremental growth. That's a phase transition.

THE NEW ENTRY-LEVEL:

WHAT THIS MEANS IF YOU'RE A CS STUDENT

Here's the part that should actually keep you up at night — in a good way.

Industry forecasts suggest that by 2030, roughly 80% of developers will be working alongside autonomous AI agents, shifting their role from someone who writes code line-by-line to someone who plans and orchestrates what the agents build.



FIGURE 2. Read that again. The job isn't disappearing. The job is moving up a layer.

Microsoft's leadership frames 2026 as an inflection point where AI stops being something that just answers questions and starts becoming a genuine collaborator — less a tool sitting on a shelf, more a colleague sitting at the desk next to you. The vision isn't agents replacing people; it's small teams operating with the leverage of much larger ones, with AI handling the grunt work while humans steer strategy.

That shift already shows up in adoption numbers. A 2025 survey of senior executives found that the vast majority — nearly four in five — said AI agents were already being adopted inside their companies. This is no longer a “someday” technology. It's a “this quarter” technology.

The developer of 2030 isn't measured by how fast they can type a for-loop — they're measured by how well they can break a vague business problem into tasks an agent can execute, verify, and correct.

In software engineering specifically, this shift is already happening — coding agents today are handling entire features and full pull-request cycles with only light human oversight.

If you're a computer science student right now, this is either the scariest sentence you'll read this year, or the most exciting one. The students who learn to direct agents — who understand systems design, tool orchestration, and how to verify AI output — will be the ones hiring managers fight over. The students who only learn to hand-write boilerplate will be competing with something that never needs a coffee break.

THE PROBLEM NOBODY'S SOLVED YET

Every powerful technology has a fault line, and agentic AI has a big one: memory and trust.

Today's AI agents have a fundamental flaw — they forget. Every new conversation starts from zero, with no memory of prior preferences, history, or decisions, essentially like hiring a brilliant assistant who wakes up with amnesia every single day.

And memory isn't the only gap. As agents get more autonomy, governance hasn't caught up. Even as adoption accelerates toward 40% of enterprise applications by year-end, the framework for governing what these agents are allowed to do is lagging badly

behind — and that governance gap is quickly becoming one of the biggest corporate risks of 2026.

This is the honest, uncomfortable center of the story: we are deploying autonomous decision-makers into production systems faster than we are building the guardrails to control them. The organizations — and the developers — who solve that problem won't just be participants in this shift. They'll be the ones who define how the next decade of software actually gets built.

WHY THIS IS THE STORY OF OUR GENERATION

Every generation of engineers gets one moment where the ground shifts under the entire profession — mainframes to PCs, PCs to the web, web to mobile, mobile to cloud. Each time, the people who adapted early didn't just survive the transition. They defined what came after it.

Agentic AI is that moment for this generation. Not a tool you use once and move on from — a fundamental rewrite of what it means to build software at all.

The question isn't whether AI agents will change your career. The global market for AI agents is already scaling at a staggering pace — from roughly \$8 billion in 2025 toward a projected quarter-trillion dollars by 2034. That train has left the station.

The only real question left is this: are you going to be the one riding it — or the one it runs past?

Federated Learning: How AI Learns Without Ever Seeing Your Data

Abdul Qayyum Tariq

What if the smartest AI models never had to see your data at all? That's the promise behind. Federated Learning (FL): training AI across phones, hospitals, and banks without ever pulling the raw data into one place. Traditional AI relies on pooling huge datasets on a single server, and that approach carries real privacy, security, and compliance risks.

The idea traces back to Google, and oddly enough, it started with something as ordinary as a keyboard. Introduced in 2016, FL sends the model out to each device instead of pulling the data in. Every participant trains locally and sends back only its updated parameters, which a central server combines using an algorithm called Federated Averaging, or FedAvg: $w(t+1) = \sum (n_k / n) \times w(t+1)^k$.

Federated learning utilizes specialized frameworks, underlying deep learning libraries, and core privacy technologies to train models across decentralized devices. Key frameworks include TensorFlow Federated (TFF), PySyft, and Flower, which work alongside privacy techniques like differential privacy and homomorphic encryption. Clients holding more data simply carry more weight in the final model — this weighted-averaging trick is what lets FedAvg train deep networks using 10 to 100 times less communication than a naive, fully distributed version of SGD.

WHY PRIVACY RULES MAKE THIS URGENT

Privacy regulation has moved from an afterthought to the rulebook that shapes how AI gets built. Hospitals can't share patient records. Banks can't exchange transaction histories. Users expect their texts to stay exactly where they typed them.

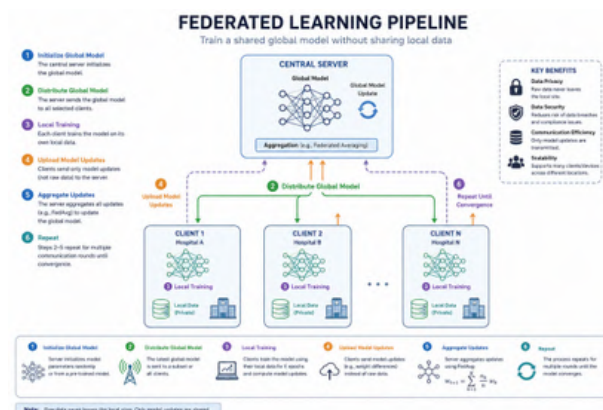


FIGURE 1. Decentralized model training via the Federated Learning pipeline while preserving local data privacy.

Cumulative GDPR fines have topped €7.1 billion since 2018, and IBM's 2025 report puts the average healthcare data breach at \$7.42 million — the costliest of any sector for 14 years running. Nineteen U.S. states now have comprehensive privacy laws on the books as of January 2026, and that list keeps growing. Against that backdrop, FL gives organizations a way to collaborate without taking on the liability of centralizing everyone's data.



FIGURE 2. Safeguarding user privacy across mobile devices, healthcare, and financial institutions.

REAL-WORLD DEPLOYMENTS

Deployment	What It Does
Google Gboard (TFF)	Trains next-word prediction across roughly 1.5 million Android devices over 3,000+ rounds — no message ever leaves the phone.
Apple QuickType & Siri	Pairs on-device learning with differential privacy for the same on-device approach.
NVIDIA Clara & FLARE	Lets hospitals like the American College of Radiology and Stanford Medicine train diagnostic models together — one mammography project used 130,000+ images across 33,000 studies without a scan changing hands.
WeBank FATE	Lets banks build fraud-detection models jointly, without ever pooling customer records.

CONCLUSION

Google's Gboard runs on TensorFlow Federated, training next-word prediction across roughly 1.5 million Android devices cumulatively over 3,000 rounds of training, all without a single message ever leaving the phone. Apple takes a similar approach in QuickType and Siri, pairing on device learning with differential privacy. In healthcare, NVIDIA's Clara and FLARE frameworks let hospitals such as the American College of Radiology and Stanford Medicine train diagnostic models together, one mammography project drew on 130,000 plus images across 33,000 studies without a single scan changing hands. And in finance, WeBank's open-source FATE toolkit lets banks build fraud-detection models jointly, without ever pooling customer records.

But it doesn't come without cost. Frequent communication between server and clients adds real overhead, and even with FedAvg's savings, large deployments still take hundreds of rounds to converge; in production systems like Gboard, a single round already needs responses from 100 to 500 devices before the model can move forward, a cost that multiplies across thousands of rounds.

Data across participants is rarely uniform either, and this so-called non-IID data can slow training and skew results. Raw data never leaves a device, but model updates themselves can occasionally leak information, which is why FL is usually paired with secure aggregation and differential privacy as an extra layer of protection. Where does that leave things? FL was never meant to replace centralized AI outright.

It's more that it's redrawing who gets a seat at the table when models are built in the first place. Real challenges around scalability and security are still active areas of research, but Federated Learning has already proven itself across healthcare, finance, and mobile computing.

REFERENCES

- H. B. McMahan et al “Communication-Efficient Learning of Deep Networks from Decentralized ” arXiv:1602.05629, 2016 (published at AISTATS, 2017).
- P. Kairouz et al., “Advances and Open Problems in Federated Learning,” Foundations and Trends in ML, 2021.

Building Software by Talking to AI

Muhammad Faraz Munir Khan

Vibe coding is a new way to create software. Instead of writing code line by line, you describe what you want in plain English to an artificial intelligence (AI), and the AI writes the code for you.

This makes building apps much faster and allows non-programmers to join the process. However, the code can have bugs, security risks, and can be hard to maintain later. This article explains what vibe coding is, its benefits, its challenges, and where it is heading. The main message is that vibe coding is a great helper, but human judgment is still needed to ensure the final product is safe and reliable.

INTRODUCTION

In early 2025, a well-known AI researcher named Andrej Karpathy posted online that he had stopped typing code himself. Instead, he just told an AI what he wanted, copied the code it gave him, and accepted everything without reading it carefully. He called this "vibe coding". His post went viral very quickly. Soon, thousands of developers started doing the same thing. Today, many companies report that a large portion of their new code is written by AI, not humans.

But is vibe coding the future of programming, or is it too risky? This article gives a simple and balanced look at what vibe coding is, why people love it, what problems it brings, and how we should use it wisely.

A BRIEF HISTORY OF PROGRAMMING

In the old days, programmers had to connect physical wires to make computers work. Later, they wrote instructions using special codes.

Then came easier languages like Python.

Each step made programming simpler and more powerful.

Vibe coding is the next big step. Now, you do not need to learn a complex programming language in full detail. You just speak naturally, and the AI turns your words into working code.

How AI Coding Tools Have Grown

AI tools that help with coding have improved very quickly. At first, they could only suggest one or two lines of code. Now, they can write whole programs, find and fix errors, and even plan large projects.

By 2025, nearly half of all developers were using these tools. Some new startups have almost all their code generated by AI.

What Vibe Coding Really Means

Simply put, vibe coding means creating software by telling an AI what you want, instead of writing the code yourself.

For example, you might say, "Build me a website where users can sign up with their email and password." The AI writes all the HTML, CSS, and JavaScript for you. You then run it, see if it works, and ask the AI to fix anything that is wrong.

The word "vibe" means you are going with the flow. You trust the AI to handle the details, and you focus on the overall look and feel of your app.

THE PROBLEMS AND RISKS

- Poor quality code. AI-generated code often works but can be messy, slow, or badly structured.
- Hard to maintain later. When you need to update the app, the code may be confusing. If you do not understand it, fixing bugs becomes very difficult.
- Security holes. AI might make mistakes, like leaving passwords exposed or allowing hackers to attack. Without careful review, these flaws stay hidden.

- You may forget how to code. If you always rely on AI, you can lose your own programming skills. This is a real danger for beginners.
- AI forgets the big picture. When conversations get long, the AI loses track of what you asked earlier. It may give answers that do not fit together.
- Blind trust. Some people accept everything the AI says without thinking. This leads to wrong or incomplete solutions.

The Real Issue: Judgment, Not Just Bugs

The biggest problem with vibe coding is not that the AI makes mistakes. The real problem is that humans need good judgment to decide if the code is correct, secure, and well-built. Vibe coding does not remove the need for judgment – it actually makes it more important. You still have to know what good code looks like, even if you did not write it.

RECENT DEVELOPMENTS

By 2026, vibe coding had already moved on. People now talk about a more disciplined version called "agentic engineering". In this approach, you do not just accept everything. You write clear instructions, ask the AI to follow them, and then test the results properly. Tools have also become more powerful. They can now handle big projects, not just small scripts. However, surveys show that while many people trust AI for simple apps, they are still wary of using it for critical systems like banking or hospital software.

Some educators have started teaching vibe coding in schools, but they always stress that students must learn the basics of programming too. The goal is to combine AI help with human understanding.

FUTURE SCOPE

Where is vibe coding going next?

- **Better AI tools** will be built specifically for conversation and collaboration, not just for typing.
- **Multiple AIs** might work together – one writes code, another tests it, another checks security, and a human oversees everything.
- **Special AI models** will be trained to write cleaner, safer, and better-documented code.
- Company rules will appear. Organizations will set policies to make sure AI-generated code is reviewed, tested, and secure before it goes live.
- Hybrid methods will become normal. You might use vibe coding for early prototypes, then switch to traditional hand-coding for the final polished product.

CONCLUSION

Vibe coding is a real revolution in software development. It makes creating apps faster and more accessible than ever before. It gives power to people who never thought they could build software.

But it is not a perfect solution. The code can be buggy, insecure, and hard to maintain. If you do not understand the code, you cannot truly own it. The greatest danger is that we lose the art of programming itself.

The smart way forward is to treat vibe coding as a powerful assistant, not a replacement for your own brain. Always review the code, test it, and learn from it. Combine the speed of AI with the careful judgment of a human.

Vibe coding is here to stay. It will only get better. But like any tool, it works best when used with skill and responsibility.

REFERENCES:

- Karpathy, A. (2025, February 2). There's a new kind of coding I call "vibe coding" . <https://x.com/karpathy/status/1886192184808149383>
- Merriam-Webster. (2025). Vibe coding. In Merriam-Webster.com dictionary. <https://www.merriam-webster.com>
- Stack Overflow. (2024). 2024 Developer Survey. <https://survey.stackoverflow.co/>
- ACM Technology Policy Council. (2025). TechBrief: AI-Assisted Coding and the Future of Software Engineering. Association for Computing Machinery.

Green LLM Inference on Edge Devices

Muhammad Faizan Javed

Large language models now support writing search, coding, translation, document analysis and many other tasks. Most public discussion about their environmental impact has focused on training. Training can use a large amount of energy, but it happens occasionally. Inference happens every time a person sends a prompt and receives an answer. For a widely used service, the energy from these repeated requests can become very large over time.

Cloud data centres remain important because they can share powerful processors among many users. They also provide strong cooling, high utilisation, and flexible capacity. A cloud-only design, however, sends data across networks, adds communication delay, and may expose private information to remote systems. Edge inference moves some or all of the work closer to the user.

The edge may be a smartphone, laptop, vehicle, factory gateway, embedded computer, or small local server. The main sustainability question is not simply “edge or cloud?” A small model running efficiently on hardware that is already available may use less energy for a routine task. A larger model running slowly on a hot, battery-powered device may use more energy and produce a worse answer. In some situations, a busy cloud accelerator powered by cleaner electricity is the better choice. A sustainable system therefore matches each task with the right model, precision, processor, and location.

WHAT GREEN EDGE INFERENCE MEANS

Edge, cloud, and hybrid execution

In cloud inference, a user’s prompt travels to a remote data centre, where a model processes it and sends the answer back.

In edge inference, the model runs on or near the user’s device. A hybrid system uses both approaches: private or simple tasks remain local, while difficult requests are sent to a larger remote model. This design is becoming more practical because compact models can perform summarisation, classification, information extraction, command interpretation, and simple question answering on limited hardware.

Where the energy goes

The electricity used by one request is not limited to mathematical calculations. The device must also move model data through memory, communicate through a network when needed, and keep hardware active while the request is processed. In simple terms, the total energy of a request includes compute energy, memory energy, network energy, and a share of idle energy.

This wider view prevents unfair comparisons. A local test is incomplete if it ignores battery losses, model downloads, or cooling. A cloud test is incomplete if it ignores network traffic and data-centre overhead. Carbon emissions also depend on the electricity source at the time and place of execution. Manufacturing matters too when a deployment requires new hardware or shortens the life of an existing device.

Why answer quality matters

Energy per token is useful, but it does not show whether the answer was helpful. A low-energy model that gives a wrong answer may cause the user to repeat the request or send it to another model. Better measures include energy per correct answer or energy per completed task. These measures connect environmental efficiency with real usefulness.

MAIN OPTIMISATION TECHNIQUES

Five techniques appear repeatedly in green edge inference. Quantisation reduces numerical precision. Pruning removes less useful parts of a model. Knowledge distillation teaches a smaller model to imitate a larger one. Context optimisation reduces the

amount of text and cached data that must be processed. Hardware-aware scheduling places each part of the work on the processor that can handle it most efficiently. Each method can help, but each also has limits.

Quantisation

Quantisation stores model values with fewer bits. For example, a model may use 8-bit or 4-bit values instead of 16-bit values. This reduces storage and memory traffic. Post-training quantisation changes a trained model without repeating full training, while quantisation-aware training prepares the model for low-precision operation during training. Methods such as GPTQ and activation-aware weight quantisation (AWQ) have made 4-bit deployment practical for many models. The important test is the complete result on the target device: a smaller model file does not guarantee lower energy when extra conversion work or unsupported instructions add overhead.

Pruning and sparsity

Pruning removes model parts that contribute little to the chosen task. These parts may include weights, neurons, attention heads, channels, or complete layers. Structured pruning removes regular blocks and is easier for common hardware to accelerate. Unstructured pruning can remove more individual weights, but the device needs efficient sparse-processing support. For this reason, measured energy and response time are more useful than simply reporting the percentage of weights removed.

Knowledge distillation

Knowledge distillation trains a smaller “student” model to imitate a larger “teacher” model. It is especially useful when the local task is narrow, such as classifying messages, extracting fields, interpreting commands, or answering questions in one subject area.

MiniLLM is one method designed to improve distillation for text generation. The main limitation is coverage: a student may work well on familiar tasks but struggle with unusual, complex, or adversarial requests.

Context and cache optimisation

Long prompts increase prefill work and enlarge the KV cache. A system can retrieve only the most relevant passages, summarise older conversation turns, remove repeated instructions, reuse common prompt prefixes, or compress the KV cache. These methods may save more energy than changing the the model when long context is the main bottleneck. They must be tested carefully because removing too much context can also remove important meaning.

Hardware-aware scheduling

Modern devices may include a CPU, GPU, and NPU. Each processor has different strengths. The prefill stage may work well on a GPU, while the decode stage may be more efficient on an NPU or another memory-friendly processor. ELLIE profiles the available processors and chooses a plan for each prompt. On its tested AI-PC platform, the study reported an average 1.8-times reduction in energy and a 1.5-times improvement in energy-delay product compared with GPU-only execution. These gains depend on the prompt, model, transfer cost, and supported hardware.

OPEN CHALLENGES AND FUTURE DIRECTIONS

The first challenge is standardised energy reporting. Without shared workloads and measurement rules, published results remain difficult to compare. The second challenge is preserving quality.

Compression can affect reasoning, factual accuracy, and safety in different ways, so smaller models need task-specific evaluation.

The third challenge is cross-layer design: model architecture, KV-cache format, compiler kernels, memory, processor placement, thermal control, and routing should be improved together.

Security and update management are equally important.

Edge devices may remain offline for long periods, which makes security patches and model removal more difficult. Future systems need signed packages, secure boot, rollback protection, provenance records, and auditable permissions. Researchers must also examine manufacturing emissions, battery ageing, and increased usage instead of measuring only electricity during a short test.

The most likely future is neither fully local nor fully cloud-based. It is a flexible system in which small models handle private and routine work near the user, while larger models are used only when necessary. The scheduler will choose the model, precision, processor, and location according to the task and the current condition of the device.

CONCLUSION

Green LLM inference on edge devices is promising, but it depends on the situation. Quantisation, pruning, distillation, context optimisation, hardware-aware

scheduling, and hybrid routing can reduce memory use, response time, and energy for well-chosen tasks. Recent systems show the value of matching the model and the two inference stages to suitable processors.

The most important lesson is that moving a model onto a device is not automatically an environmental success. Sustainability depends on the complete system: answer quality, compute and memory energy, network traffic, hardware utilisation, electricity source, temperature, battery effects, device lifetime, and manufacturing impact. A practical green design measures these factors, keeps suitable tasks local, and sends a request to a larger system only when the additional capability is genuinely needed.



FIGURE 1. Sustainable and energy-efficient LLM inference powering green edge computing.

REFERENCES

- Y. Mao, X. Yu, K. Huang, Y.-J. A. Zhang, and J. Zhang, “Green edge AI: A contemporary survey,” *Proceedings of the IEEE*, vol. 112, no. 7, pp. 880–911, 2024, doi: 10.1109/JPROC.2024.3437365.
- O. Friha, M. A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, and N. Ghoualmi-Zine, “LLM-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness,” *IEEE Open Journal of the Communications Society*, vol. 5, pp. 5799–5856, 2024, doi: 10.1109/OJCOMS.2024.3456549.
- Z. Lin, G. Qu, Q. Chen, X. Chen, Z. Chen, and K. Huang, “Pushing large language models to the 6G edge: Vision, challenges, and opportunities,” *IEEE Communications Magazine*, vol. 63, no. 9, pp. 52–59, 2025.
- S. O. Semerikov et al., “LLM on the edge: The new frontier,” *CEUR Workshop Proceedings*, vol. 3943, pp. 137–161, 2025.

Physical AI: What Happens When AI Agents Enter the Real World?

Muhammad Rafay Shafqat

“Prometheus stole fire from the gods and gave it to humanity. In the modern retelling, we are giving intelligence to machines, and asking them to carry it into the physical world.”

WHAT MAKES AI PHYSICAL?

For most of the modern AI era, intelligence lived behind glass. Systems classified images, predicted words, generated code, and answered questions, while their errors largely remained inside a screen. Physical AI changes that boundary by connecting intelligence to cameras, microphones, tactile sensors, wheels, robotic arms, vehicles, drones, and the environments where people live and work. Once a system can move, touch, carry, steer, or manipulate, a plausible answer is no longer enough. It must understand geometry, timing, friction, uncertainty, and human behavior, then act before the world changes again. Physical AI is therefore not simply AI placed inside a robot. It is intelligence engineered for a world that pushes back.

Physical AI refers to autonomous systems that perceive, reason about, and act in the physical world through a continuous sensing-and-action loop. A complete system interprets observations, represents the scene, plans an action, executes it through hardware, and measures what actually happened. This is why a foundation model alone is not a physical AI system. A vision-language model may recognize a mug and understand “place it on the shelf,”

but the system must still estimate the mug’s position, check reachability, plan a collision-free motion, select a safe grip, control the arm, detect slippage, and verify successful placement.

PERCEPTION, WORLD MODELS, AND PLANNING

A physical agent never sees the world directly; it receives measurements. Cameras produce pixels, lidar produces distances, microphones produce waveforms, and force sensors report pressure. Perception software must convert these noisy and incomplete signals into a usable situation: what is present, what is moving, where objects and people are, and which actions are physically possible. Modern multimodal models can connect visual observations with language and prior knowledge, but recognition is only the first step. The system must also estimate position, track objects over time, infer hidden state, and account for changing lighting, occlusion, drift, and unpredictable human movement.

Before acting, the machine also needs a world model: an internal representation used to anticipate consequences. Some world models are geometric and explicit, using maps, object locations, motion equations, and known constraints. Others are learned from data and predict future observations or states. In practice, hybrid systems are often strongest because learned models provide flexible understanding while classical planning and control enforce geometry, timing, and safety.

The model does not need to reproduce reality perfectly, but it must be accurate enough for the decision. Small errors can compound when a surface is slippery, an object is heavier than expected, or a person unexpectedly enters the workspace.

THE PHYSICAL AI STACK

Layer	Primary role
Perception	Turn raw sensor signals into usable observations
World model	Represent the scene and predict how it may change
Reasoning and planning	Translate a goal into actions
Agent harness	Manage the autonomous loop
Control	Convert plans into precise movement
Safety and evaluation	Decide whether behavior is acceptable

THE AGENT HARNESS AND CONTROL

The reasoning model receives attention, but the agent harness often determines whether autonomy remains coherent. The harness is the runtime around the model. It selects context, exposes tools, summarizes sensor observations, stores memory, validates proposed actions, tracks failed attempts, enforces permissions, requests human approval, and decides when a task should stop.

Autonomy is therefore a system property. A powerful model connected to unreliable tools, stale memory, or vague stopping rules may perform worse than a smaller model inside a disciplined harness.

Planning must also be separated from control. “Pick up the bottle” is a task-level instruction, but the machine still needs a collision-free arm path, wrist orientation, grip force, speed profile, and recovery response. High-level models may reason in seconds, while low-level control loops compare desired and measured movement in milliseconds. End-to-end vision-language-action models can reduce hand-built pipelines and improve generality, but practical systems still require hardware limits, monitoring, feedback control, and recovery logic. Fluency at the task level does not eliminate physics at the actuator level.

SIMULATION

Training only in the real world is slow, expensive, and sometimes dangerous. Simulation lets robots practice thousands of grasps, allows autonomous vehicles to encounter rare hazards, and enables digital twins of factories or warehouses. It also makes evaluation repeatable across models, software versions, sensors, lighting, friction, latency, and human movement. Yet simulated success is not real-world proof. The sim-to-real gap remains because simulators simplify materials, contacts, sensors, and people. Domain randomization, realistic noise, real-world fine-tuning, and continual calibration can narrow the gap, but not remove it.

A physical agent must be evaluated across an entire trajectory rather than only by its final answer. Evaluation should ask whether the task was completed, whether constraints were obeyed, how close the system came to

collision, whether it recovered from uncertainty, and whether it stopped safely when sensors disagreed. Component tests measure perception, pose estimation, planning, and controller stability. Scenario and stress tests introduce occlusion, sensor failure, delays, unusual objects, adversarial instructions, and unpredictable human movement. Field trials then measure performance in the actual operating environment. Execution traces matter because a successful task can hide a dangerous near miss, while a failed task may demonstrate the correct decision to stop

SAFETY, SECURITY, AND HUMAN AUTHORITY

When AI controls hardware, mistakes gain momentum and mass. Safety must cover perception failures, insecure commands, planning errors, network delays, hardware faults, and interactions with people.

Robust designs use layers: model instructions, harness-level tool restrictions, human approval, planner-enforced keep-out zones, independent speed or contact monitors, mechanical limits, and emergency stops. The system must also express uncertainty by slowing down, seeking another view, asking for clarification, choosing a conservative action, or stopping completely.

Security is inseparable from safety. A language-guided robot can be manipulated through malicious prompts, corrupted sensor inputs, compromised tools, or poisoned memory.

An attacker may not need direct motor control; altering what the agent believes about the room may be enough. Human judgment therefore does not disappear. People define goals, tools, permissions, evaluation scenarios, intervention procedures, maintenance practices, and acceptable risk.

For high-stakes systems, the question is not merely whether a human is “in the loop,” but where authority enters, how quickly intervention is possible, what evidence the operator receives, and whether the machine fails safely when no one responds.

WHERE PHYSICAL AI IS TAKING SHAPE

Physical AI is emerging in robots and autonomous vehicles, but also in manufacturing, warehouses, healthcare, agriculture, homes, and smart spaces. Robots can adapt to variation rather than repeat one fixed motion; mobile systems can navigate changing layouts; assistive machines can support mobility, rehabilitation, logistics, and monitoring; and distributed cameras, edge devices, and machines can coordinate as one intelligent environment.

These capabilities also expand the surface for failure, surveillance, and cyberattack.

THE OLD DREAM, NEW RESPONSIBILITIES

Prometheus is remembered not only for giving humanity fire, but for giving us a power whose usefulness could not be separated from its consequences.

Physical AI carries the same tension. The challenge is not simply to build machines that reason, but complete systems that perceive uncertainty, predict consequences, act within constraints, expose their decisions, and prove reliability through rigorous evaluation.

The next generation of AI will move into roads, factories, hospitals, homes, and public spaces. The question is no longer whether machines can be given intelligence, but whether we can engineer the judgment, restraint, and accountability that must travel with it.

REFERENCES AND FURTHER READING

1. NVIDIA, “What Is Physical AI?” NVIDIA Glossary, accessed July 2026.
2. Google DeepMind, “Gemini Robotics,” model overview and technical resources, accessed July 2026.
3. Google DeepMind, “Gemini Robotics On-Device brings AI to local robotic devices,” June 24, 2025.
4. X. Long et al., “Learning Embodied Intelligence from Physical Simulators and World Models,” arXiv:2507.00917, 2025.
5. W. Liang et al., “Large Model Empowered Embodied AI,” arXiv:2508.10399, 2025.
6. X. Li et al., “Safety in Embodied AI,” arXiv:2605.02900, 2026.

IEEE EVENTS



WEBINAR

Our webinar series brought together experienced professionals to share their expertise across diverse domains of computing.



KNOWLEDGE SHARED.IDEAS INSPIRED.IMPECT DELIVERED

SPOTLIGHTS

A glimpse into the most memorable moments of our events — where ideas sparked, skills grew, and connections were made.



EVERY EVENT, ACHIEVEMENTS, AND EXPERIENCES TELLS A STORY
THAT DEFINED OUR **IEEE COMPUTER SOCIETY** JOURNEY.

IEEE TECH COMPETITION

ITC 2.0

INNOVATE • TRANSFORM • COMPETE

ITC 2.0 is our flagship annual event that brings together passionate tech enthusiasts to solve, compete, and create the future. From challenging technical battles to thrilling gaming arenas, ITC is where innovation meets excitement. One event. Infinite possibilities.

ONE STAGE.
COUNTLESS MINS.
ENDLESS **IMPACT**

 **170+**
PARTICIPANTS

 **6**
WINNERS

 **6**
RUNNERUP

 **3**
GAMMING
COMPETITIONS

 **3**
TECH
COMPETITIONS

ONE WEEK. ENDLESS **LEARNING**, MEANINGFUL **CONNECTIONS**,
AND REMARKABLE **ACHIEVEMENTS**.

•• BIGGER STAGE. BETTER COMEPETITION. **STRONER COMMUNITY**



IEEE
COMPUTER
SOCIETY

UET Narowal Chapter | نارووال

WEBINAR

Spotlights

ITC 2.0

Closing
Ceremony

ITC 2.0 TECH COMPETITIONS

Where ideas meet challenge and skills turn into impact. ITC 2.0 witnessed three intense tech competitions that pushed participants to think smarter, code faster and solve better.



01 BUG BOUNTY

Participants analyzed code, identified hidden bugs, and demonstrated their debugging and problem-solving skills.



02 SPEED PROGRAMMING

Time, logic and precision—participants raced against the clock to solve as many problems as possible in the fastest time.



03 CTRL+ QUIZ

Brains on! A test of knowledge, awareness and quick thinking across the world of technology and beyond.



ITC 2.0 GAMING ARENA

Our gaming arena brought together top gamers, battled, strategized, and entertained the crowd with unmatched energy.



01 TEKKEN 7



Fast-paced battles where precision and quick reflexes determined victory



02 ESCAPE ROOM



A challenge of logic, teamwork, and thinking under the pressure of time.



03 PUBG

Intense battle where survival, and tactical decision-making took center stage.



CLOSING CEREMONY

A celebration of determination, teamwork, and excellence. Honoring the champions who rose above challenges and made ITC 2.0 unforgettable.



MEET OUR CHAMPIONS



OUR ALUMNI 2025-2026

THE JOURNEY ENDS, THE LEGACY CONTINUES.



“
Every challenge
created an
opportunity. Every
participant created a
story. Every moment
became a memory.”

**THANK
YOU**

to everyone who
made ITC 2.0
a grand success!





UNTIL NEXT
BYTE



THANK YOU FOR BEING A PART.

COMPUTING SPECTRUM

IEEE Uet Narowal Student Branch

IEEE UET Narowal Student Branch has emerged as a dynamic and forward-looking platform dedicated to promoting technical excellence, professional development, and leadership among students. The branch serves as a bridge between academic learning and real world engineering practices, empowering students to grow beyond the classroom.

Leadership & Vision

IEEE UET Narowal Student Branch has emerged as a dynamic and forward-looking platform dedicated to promoting technical excellence, professional development, and leadership among students. The branch serves as a bridge between academic learning and real world engineering practices, empowering students to grow beyond the classroom.

Key Achievements & Milestones

Throughout the term, the branch successfully organized a series of technical workshops, seminars, awareness sessions, and professional development activities. The activities include the **Funding of \$104** – IEEE Student Branch Rebate ComSoc Membership Development Support Grant (MDSG) PES HPSBCP Grant, and the IET spark debate competition that was won by iee narowal brilliant minds.

The milestones we achieved that includes the publishing of research papers that were presented on umt ssc by one of our branches chairpersons and we how can we forget the **TLS series win**.

These 4 milestones **funding of \$500**, IET spark debate competition, paper publication and TLS series win were a major set up for IEEE UET NAROWAL.

Member Engagment & Capacity Building

Active participation from members remained one of our greatest strengths. By involving students in planning, execution, and leadership roles, the branch provided hands on experience in teamwork, event management, and decision making. This inclusive approach enhanced confidence and nurtured future leaders.

Academic, Professional & Community Impact

The branch played a vital role in enhancing students' exposure to industry relevant knowledge and professional standards. Alongside technical advancement, community-oriented initiatives reinforced ethical responsibility, publishing many research papers and social awareness, reflecting IEEE's commitment to serving humanity through technology.

Collaborating & Recognition

Collaboration with university departments, IEEE bodies, and professional communities strengthened the branch's outreach and credibility. These partnerships enabled the successful delivery of quality events and positioned IEEE UET Narowal Student Branch as an active and reliable contributor within the IEEE network.

Through collaborations with university departments and professional bodies, the chapter expanded its outreach and credibility.

Some mega collaborations include:

IEEE UNSB x UMT

IEEE UNSB x UCP

IEEE UNSB x IEEE FSD

IEEE UNSB x IEEE RCET

IEEE UNSB x IEEE KSK

IEEE UNSB x IEEE Comsats

And there's many to mention. These are collaborations only done in these 4-5 months. These partnerships played a vital role in delivering high-quality events and gaining recognition within the academic and professional community.

Collaborations and core community is impacting massively

Building on the achievements of this term, the IEEE UET Narowal Student Branch aims to expand technical programs, strengthen industry linkages, and promote research and innovation. With continued dedication and collective effort, the branch is well-positioned to achieve greater milestones in the years ahead.

- Planning to boost the membership plans
- Arranging the industrial tours
- Beneficial seminars
- Technical series are loading
- Big collaborations

Message from the Chairperson

The success of IEEE UET Narowal Student Branch is a result of teamwork, commitment, and shared vision. I extend my sincere appreciation to the executive body, volunteers, and members whose efforts transformed goals into achievements. Together, we continue to grow, lead, and innovate.

Chairperson

IEEE UET Narowal Student branch

COMPUTING SPECTRUM



Stay in the loop – follow us on social Media!

